Name: Razel Navales

Laboratory Experiment # 004: Transmission Technologies

Multiplexing is the set of techniques that allows the simultaneous transmission of multiple signals across a single data link. **Spreading** (Spread Spectrum) is a technique that expands the bandwidth of a signal to prevent jamming, mitigate interference, and allow multiple access.

In this notebook, we will simulate:

1. Frequency-Division Multiplexing (FDM)
2. Time-Division Multiplexing (TDM)
3. Direct Sequence Spread Spectrum (DSSS)

Let's start by importing the necessary Python libraries.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

# Set up global plot styling
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['lines.linewidth'] = 2
```

1. Frequency-Division Multiplexing (FDM)

FDM assigns different carrier frequencies to different signals so they can share the same medium without overlapping. It is an analog multiplexing technique widely used in radio and television broadcasting.

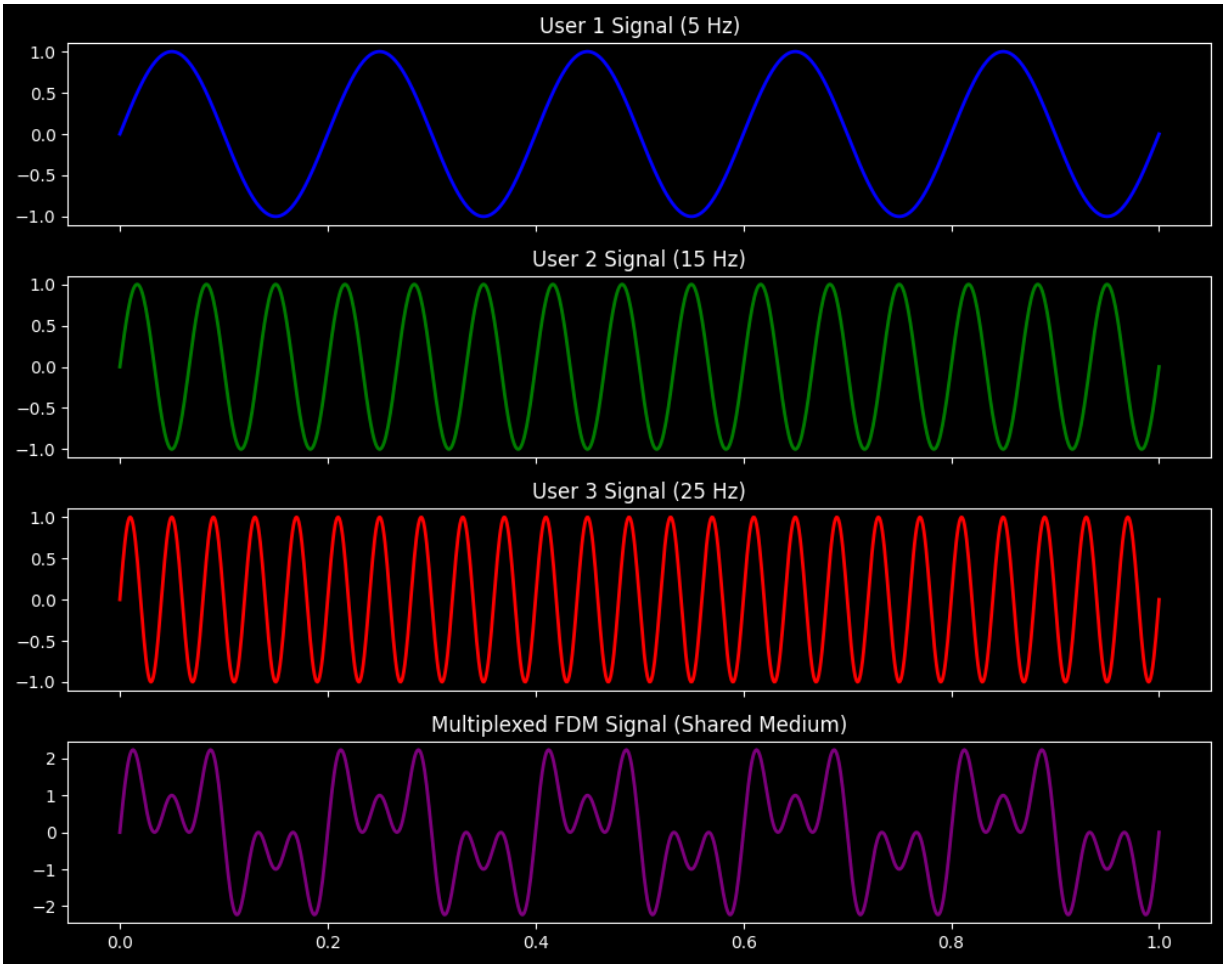
Math Representation: The composite signal $S(t)$ is the sum of modulated signals:

$$S(t) = \sum_{i=1}^{n} m_i(t) \cdot \cos(2\pi f_{ci} t)$$

Where f_{ci} is the distinct carrier frequency for user i.

In [2]:
```
# Time array
t = np.linspace(0, 1, 1000)

# Create 3 distinct base signals (simulating different users)
sig1 = np.sin(2 * np.pi * 5 * t)  # User 1: 5 Hz
sig2 = np.sin(2 * np.pi * 15 * t) # User 2: 15 Hz
sig3 = np.sin(2 * np.pi * 25 * t) # User 3: 25 Hz
```

```
# Multiplexing: Combining them into a single medium
fdm_signal = sig1 + sig2 + sig3

# Plotting the FDM process
fig, axs = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
axs[0].plot(t, sig1, color='blue')
axs[0].set_title('User 1 Signal (5 Hz)')

axs[1].plot(t, sig2, color='green')
axs[1].set_title('User 2 Signal (15 Hz)')

axs[2].plot(t, sig3, color='red')
axs[2].set_title('User 3 Signal (25 Hz)')

axs[3].plot(t, fdm_signal, color='purple')
axs[3].set_title('Multiplexed FDM Signal (Shared Medium)')

plt.tight_layout()
plt.show()
```

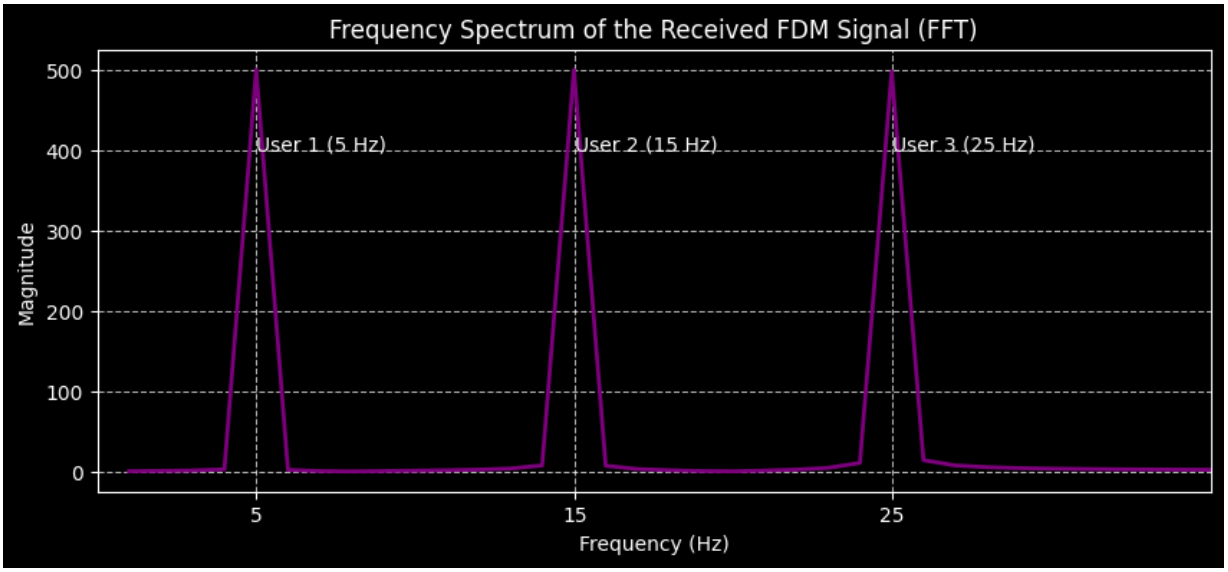


2. Demultiplexing Frequency-Division Multiplexing (FDM)

At the receiver end, the FDM signal must be separated back into its original components. This is typically done using **Bandpass Filters** that isolate specific frequency ranges.

In digital signal processing, we can use the Fast Fourier Transform (FFT) to analyze the frequencies present in the received composite signal and verify that our original signals (5 Hz, 15 Hz, and 25 Hz) are perfectly preserved.

In [3]:
```
# Apply Fast Fourier Transform (FFT) to the composite FDM signal
n = len(fdm_signal)
fft_result = np.fft.fft(fdm_signal)
frequencies = np.fft.fftfreq(n, d=(t[1] - t[0]))

# We only care about the positive frequencies for visualization
pos_mask = frequencies > 0
freqs_pos = frequencies[pos_mask]
fft_mag_pos = np.abs(fft_result)[pos_mask]

# Plotting the Frequency Spectrum
plt.figure(figsize=(10, 4))
plt.plot(freqs_pos, fft_mag_pos, color='purple')
plt.title("Frequency Spectrum of the Received FDM Signal (FFT)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.xlim(0, 35)
plt.xticks([5, 15, 25])
plt.grid(True, linestyle='--', alpha=0.7)
plt.annotate('User 1 (5 Hz)', xy=(5, max(fft_mag_pos)), xytext=(5, max(fft_mag_pos)
plt.annotate('User 2 (15 Hz)', xy=(15, max(fft_mag_pos)), xytext=(15, max(fft_mag_p
plt.annotate('User 3 (25 Hz)', xy=(25, max(fft_mag_pos)), xytext=(25, max(fft_mag_p
plt.show()
```



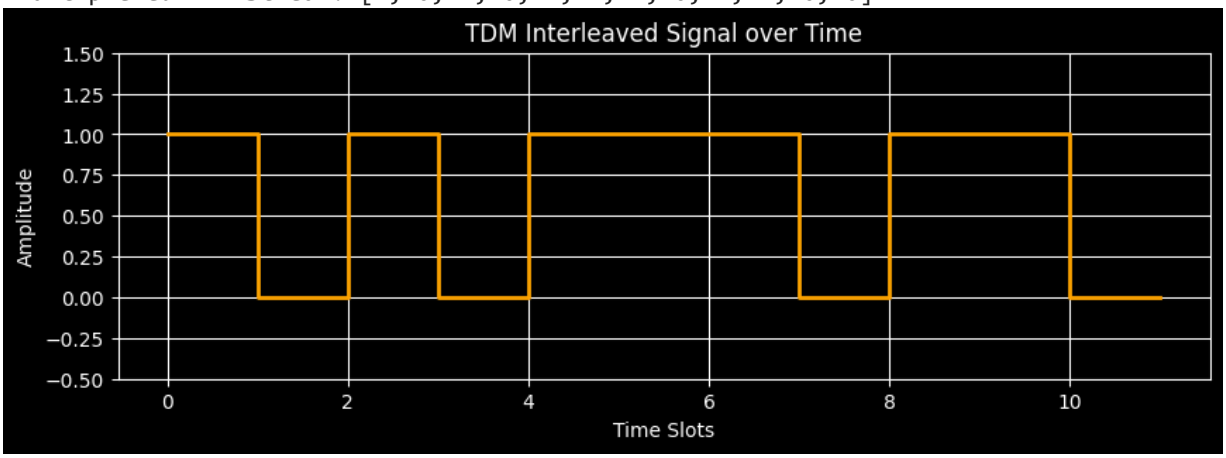
3. Time-Division Multiplexing (TDM)

TDM is a digital process that allows several connections to share the high bandwidth of a link. Instead of sharing frequency, users share *time*. Each user is given a brief time slot in a

round-robin fashion.

In [4]:
```
# 3 digital data streams (4 bits each)
user1_bits = [1, 0, 1, 1]
user2_bits = [0, 1, 0, 0]
user3_bits = [1, 1, 1, 0]

# Multiplexer (Interleaving bits)
tdm_frame = []
for i in range(len(user1_bits)):
    tdm_frame.extend([user1_bits[i], user2_bits[i], user3_bits[i]])

print(f"User 1 Data: {user1_bits}")
print(f"User 2 Data: {user2_bits}")
print(f"User 3 Data: {user3_bits}")
print("-" * 30)
print(f"Multiplexed TDM Stream: {tdm_frame}")

# Plotting the TDM stream
plt.figure(figsize=(10, 3))
plt.step(range(len(tdm_frame)), tdm_frame, where='post', color='orange')
plt.ylim(-0.5, 1.5)
plt.title("TDM Interleaved Signal over Time")
plt.xlabel("Time Slots")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

```
User 1 Data: [1, 0, 1, 1]
User 2 Data: [0, 1, 0, 0]
User 3 Data: [1, 1, 1, 0]
------------------------------
Multiplexed TDM Stream: [1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0]
```



4. Demultiplexing Time-Division Multiplexing (TDM)

To demultiplex a TDM signal, the receiver must be perfectly synchronized with the transmitter. It uses a synchronized switch to route the incoming interleaved bits back to their respective user channels based on their time slot index.

In [5]:
```
# Demultiplexing: Reversing the interleave process
# We know there are 3 users, so we extract every 3rd bit starting from a specific o
rx_user1 = tdm_frame[0::3]
rx_user2 = tdm_frame[1::3]
rx_user3 = tdm_frame[2::3]

print(f"Received TDM Stream: {tdm_frame}")
print("-" * 30)
print(f"Recovered User 1: {rx_user1}  (Match? {rx_user1 == user1_bits})")
print(f"Recovered User 2: {rx_user2}  (Match? {rx_user2 == user2_bits})")
print(f"Recovered User 3: {rx_user3}  (Match? {rx_user3 == user3_bits})")
```

```
Received TDM Stream: [1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0]
------------------------------
Recovered User 1: [1, 0, 1, 1]  (Match? True)
Recovered User 2: [0, 1, 0, 0]  (Match? True)
Recovered User 3: [1, 1, 1, 0]  (Match? True)
```

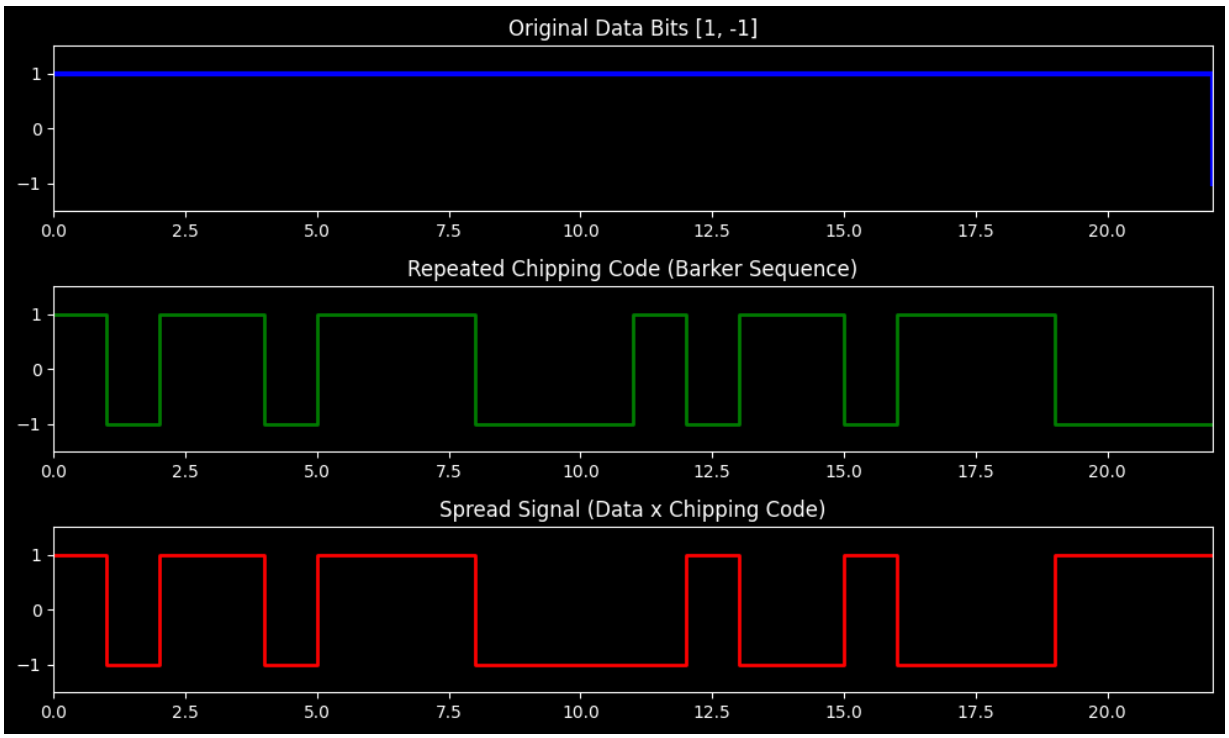
5. Spread Spectrum: Direct Sequence Spread Spectrum (DSSS)

Instead of squeezing a signal into a narrow band, Spread Spectrum intentionally spreads the signal over a wider frequency band. This provides privacy and resistance to jamming.

In **DSSS**, we replace each data bit with an n-bit spreading code (a chipping code). A common example is the 11-bit Barker code used in 802.11b Wi-Fi.

In [6]:
```
# Original Data: [1, 0] represented as [1, -1] for bipolar encoding
data = np.array([1, -1])

# 11-bit Barker Code (Chipping code)
chipping_code = np.array([1, -1, 1, 1, -1, 1, 1, 1, -1, -1, -1])

# Spreading the signal using Kronecker product
spread_signal = np.kron(data, chipping_code)

# Plotting
fig, axs = plt.subplots(3, 1, figsize=(10, 6), sharex=False)

# Original data plot
axs[0].step([0, 11, 22], [1, 1, -1], where='post', color='blue', linewidth=3)
axs[0].set_title('Original Data Bits [1, -1]')
axs[0].set_xlim(0, 22)
```

```
axs[0].set_ylim(-1.5, 1.5)

# Chipping code plot
chipping_repeated = np.tile(chipping_code, 2)
axs[1].step(range(len(chipping_repeated) + 1), np.append(chipping_repeated, chippin
axs[1].set_title('Repeated Chipping Code (Barker Sequence)')
axs[1].set_xlim(0, 22)
axs[1].set_ylim(-1.5, 1.5)

# Transmitted spread signal
axs[2].step(range(len(spread_signal) + 1), np.append(spread_signal, spread_signal[-
axs[2].set_title('Spread Signal (Data x Chipping Code)')
axs[2].set_xlim(0, 22)
axs[2].set_ylim(-1.5, 1.5)

plt.tight_layout()
plt.show()
```



6. Despreading Direct Sequence Spread Spectrum (DSSS)

To recover the original data from a DSSS signal, the receiver must know the exact chipping code (Barker sequence) used by the transmitter.

The receiver multiplies the incoming spread signal by the synchronized chipping code and sums the result over the bit period (this is known as calculating the correlation).

- If the sum is highly positive, the original bit was a 1.
- If the sum is highly negative, the original bit was a -1 (or 0).

In [7]:
```
# The receiver uses the exact same 11-bit Barker code
rx_chipping_code = np.array([1, -1, 1, 1, -1, 1, 1, 1, -1, -1, -1])

recovered_data = []

# Process the received signal in chunks of 11 chips
chunk_size = len(rx_chipping_code)

for i in range(0, len(spread_signal), chunk_size):
    # Extract the chunk corresponding to one original data bit
    chunk = spread_signal[i:i + chunk_size]

    # Despreading: Multiply the received chunk by the chipping code and sum
    correlation = np.sum(chunk * rx_chipping_code)

    # Decision threshold
    if correlation > 0:
        recovered_data.append(1)
    else:
        recovered_data.append(-1)

print(f"Transmitted Spread Signal:\n{spread_signal}\n")
print(f"Receiver correlation sums: {[np.sum(spread_signal[i:i+11] * rx_chipping_cod
print(f"Recovered Data: {recovered_data}")
print(f"Original Data:  {list(data)}")
print(f"Successful transmission? {np.array_equal(recovered_data, data)}")
```

```
Transmitted Spread Signal:
[ 1 -1  1  1 -1  1  1  1 -1 -1 -1 -1  1 -1 -1  1 -1 -1 -1  1  1  1]

Receiver correlation sums: [np.int64(11), np.int64(-11)]
Recovered Data: [1, -1]
Original Data:  [np.int64(1), np.int64(-1)]
Successful transmission? True
```